Real-Time AI Prediction Platform with Automated MLOps Pipeline

Technical Documentation

Prepared by: Navodi Senevirathne

Project type: Individual task

Project duration: 2 October 2026 – 16 October 2026 (2 weeks)

Submission date: 13 October 2026

GitHub repository	https://github.com/NavodiSenevirathne/AI_prediction_platform
Live prediction API	https://churn-prediction-api-uw5o.onrender.com/docs
Monitoring dashboard	https://churn-monitoring-dashboard.onrender.com
Experiment tracking	https://dagshub.com/NavodiSenevirathne/AI_prediction_platform/experiments

Contents

1. Overview 1
   - Business problem 1
   - Objectives and how each is met 1
   - Live system 1
2. System architecture 2
   - Components 2
   - Key design decisions 3
3. Data 4
   - Features 4
   - How churn is generated 4
   - Drift simulation 4
   - Data validation 4
4. Model 5
   - Pipeline 5
   - Training and evaluation 5
   - Production model registry (models/) 5
5. Experiment tracking 6
6. Real-time prediction API 8
   - Endpoints 8
   - Example 8
   - Input validation 8
   - Prediction logging 8
7. Data drift monitoring 10
   - Population Stability Index (PSI) 10
   - Statistical tests (reported alongside PSI) 10
   - Thresholds 10
8. Automated retraining pipeline 11
   - Steps 11
   - Retraining triggers 11
   - Outputs of every run 12
   - Run locally 12
9. CI/CD 13
   - Image tags 13
   - Repository settings 13
10. Deployment 15

Docker images 15
Render services 15
Running the whole platform locally 16
11. Monitoring dashboard 17
12. Testing 20
13. Results 21
End-to-end run, 13 Oct 2026 (Sri Lanka time) ..... **Error! Bookmark not defined.**
Retraining decision 21
Model versions 22
Other checks 23
14. Limitations and future work 24
Limitations 24
Future work 24
Appendix 25
A. Repository structure 25
B. Running locally (Windows PowerShell) 25
C. Technology stack 25

1. Overview

The platform serves real-time customer-churn predictions and keeps its own model up to date: it detects data drift, retrains, and redeploys a better model with no manual step. On 13 Oct 2026 it promoted model v1 (ROC-AUC 0.864) to v2 automatically after detecting drift in 3 of 9 features.

Business problem

A telecom company wants to know, the moment a customer interacts with any channel, how likely that customer is to cancel (churn), so retention offers can be targeted. Customer behaviour changes over time (new pricing, more fibre customers, heavier usage), so a model trained once goes stale. The platform must notice that change and update the model by itself.

Objectives and how each is met

Objective	Implementation
Real-time predictions	FastAPI service, POST /predict and /predict/batch, about 10 ms per request
Automated model retraining	Daily GitHub Actions workflow with a champion/challenger gate
Continuous deployment	CD workflow builds versioned Docker images and triggers a Render deploy hook, then verifies the live version
Experiment tracking	MLflow hosted on DagsHub: parameters, metrics, drift reports, models, Model Registry
Data drift monitoring	PSI with KS and chi-squared tests per feature, on live traffic and inside the pipeline
Cloud deployment	Two Docker web services on Render (API and dashboard)
CI/CD integration	Three workflows: CI, CD and automated retraining
Monitoring dashboard	Streamlit app: traffic, latency, drift, model history, live scoring

Live system

Component	Link
Prediction API (interactive docs)	https://churn-prediction-api-uw5o.onrender.com/docs
Monitoring dashboard	https://churn-monitoring-dashboard.onrender.com
Model metadata and version history	https://churn-prediction-api-uw5o.onrender.com/model/info
Experiment tracking	https://dagshub.com/NavodiSenevirathne/AI_prediction_platform/experiments
Source code	https://github.com/NavodiSenevirathne/AI_prediction_platform

The services run on Render’s free tier and sleep after 15 minutes without traffic; the first request after that takes about a minute.

2. System architecture

The system has three layers: production serving on Render, MLOps automation in GitHub Actions, and storage plus tracking in GitHub, GitHub Container Registry and DagsHub. No component needs a person to move a model from training to production.

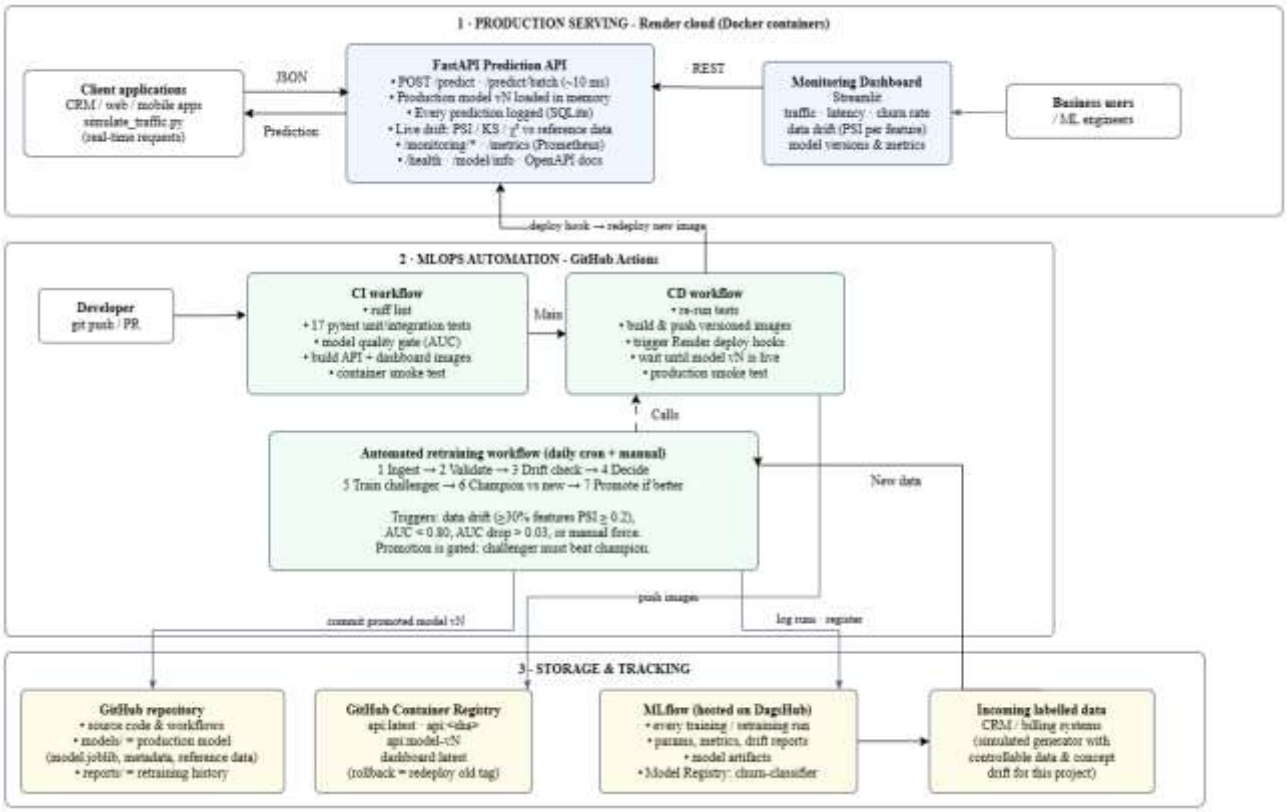


Figure 1. System architecture: serving, automation and storage layers

Components

Layer	Component	Technology	Responsibility
Serving	Prediction API	FastAPI, Uvicorn, Docker	Validates input, predicts with the in-memory model, logs every prediction, exposes monitoring endpoints
Serving	Prediction log	SQLite inside the API container	Stores features, probability, latency and model version per request
Serving	Monitoring dashboard	Streamlit, Docker	Shows traffic, latency, drift, model history; sends test traffic
Automation	CI workflow	GitHub Actions	Lint, tests, quality gate, image build, container smoke test
Automation	CD workflow	GitHub Actions, Render deploy hooks	Builds and pushes images, deploys, verifies the live model version
Automation	Retraining workflow	GitHub Actions (daily cron)	Drift check, retrain, compare, promote, then calls CD
Storage	GitHub repository	Git	Code, workflows, the production model (models/), retraining reports
Storage	Container registry	GitHub Container Registry (GHCR)	Versioned images: latest, git SHA, model-vN

Tracking	MLflow on DagsHub	MLflow 3	Runs, metrics, drift reports, model artifacts, Model Registry

Key design decisions

- **Champion/challenger gating.** A new model is deployed only if it beats the current model on the same fresh hold-out data, so retraining cannot make production worse.
- **Model baked into the image.** Each image is an immutable model version, the API does not depend on MLflow at start-up, and a rollback is a redeploy of an older image.
- **Deploy hooks instead of auto-deploy.** Render auto-deploy is off; Render deploys only when CD calls the hook after tests pass.
- **GitHub Actions as orchestrator.** It is free, versioned with the code, and shows the whole loop in one place. In a larger company the same steps map onto Airflow, Prefect or Kubeflow.

3. Data

The platform uses a simulated customer data source with a controllable drift level, because a fixed dataset never changes and would give the system nothing to detect or adapt to. In a real company this feed would come from CRM and billing systems; `ingest()` can read a real CSV instead with `--data file.csv`.

Features

Feature	Type	Description
age	integer	Customer age, 18–85
tenure_months	integer	Months as a customer, 0–72
monthly_charges	float	Current monthly bill
total_charges	float	Total billed so far
num_support_calls	integer	Support calls in the last period
avg_daily_usage_gb	float	Average daily data usage
contract_type	category	month_to_month, one_year, two_year
internet_service	category	dsl, fiber, none
payment_method	category	credit_card, bank_transfer, electronic_check
churn (target)	0 / 1	1 = the customer left

How churn is generated

`generate_customers(n, drift, seed)` in `src/data/generate.py` draws each customer's churn from a logistic model with realistic effects. Month-to-month contracts, short tenure, many support calls, high charges, fibre internet and electronic-check payment raise the churn probability; two-year contracts and older age lower it. The base population churns at about 32%.

Drift simulation

The `drift` parameter runs from 0 (the original population) to 1 (a strongly changed market) and produces two kinds of drift:

- **Covariate drift:** more month-to-month and fibre customers, prices up to 15 higher, more support calls, heavier usage, younger customers and shorter tenure.
- **Concept drift:** heavy users (more than 5 GB a day) start churning more, so the relationship between features and churn also changes.

Scheduled retraining runs use a drift level that grows with the date: 0 on 2 Oct 2026, rising to 1.0 after 21 days. Manual runs can set any level.

Data validation

Before training, `validate()` stops the pipeline when any check fails:

- All 9 feature columns and the target are present
- At least 500 rows
- No column more than 5% missing
- No negative numeric values
- No unknown category values
- The target is binary (0/1)

4. Model

The model is a scikit-learn pipeline (preprocessing plus a HistGradientBoosting classifier) trained on 16,000 customers and evaluated on 4,000 held-out customers; v1 reached ROC-AUC 0.864.

Pipeline

1. StandardScaler on the 6 numeric features.
2. OneHotEncoder on the 3 categorical features, with fixed category lists and unknown values ignored.
3. HistGradientBoostingClassifier: learning rate 0.06, 250 iterations, 31 leaf nodes, L2 regularisation 1.0, random state 42.

Preprocessing and model are saved as one pipeline object, so the API applies exactly the transformations used in training (no training/serving skew).

Gradient boosting was chosen because it is strong on tabular data, fast to train and to predict, and handles mixed feature types well.

Training and evaluation

- 20,000 customers generated with drift 0, split 80/20 stratified on churn.
- Decision threshold 0.5. Risk levels: low below 0.4, medium 0.4–0.7, high 0.7 and above.
- ROC-AUC is the main metric: it measures how well the model ranks churners above non-churners and is not distorted by the 32% churn rate

Metric (v1, held-out test set)	**Value**
ROC-AUC	0.864
F1	0.674
Precision	0.711
Recall	0.641
Accuracy	0.804
Log loss	0.421

Production model registry (models/)

File	**Content**
model.joblib	The fitted pipeline
metadata.json	Version, training time, parameters, metrics, MLflow run id, data source, promotion reason, history of the last 25 versions
reference_sample.csv	3,000 rows of the training data, the baseline for drift detection

This folder holds only the model currently in production and is committed to Git. MLflow keeps every candidate model and registers each promoted one.

5. Experiment tracking

Every training and retraining run is logged to MLflow hosted on DagsHub, and every promoted model is registered as a new version of churn-classifier in the MLflow Model Registry.

What is logged	Examples
Parameters	Model hyperparameters, simulated drift level, data source, row count, seed
Metrics	Test metrics; champion and challenger ROC-AUC, F1, precision, recall; share of drifted features; champion AUC drop
Artifacts	The model (with input example and signature), drift_report.json
Tags	Git commit SHA, stage (initial training or retraining), decision, promoted flag

src/training/tracking.py wraps MLflow so that tracking never breaks training:

- With MLFLOW_TRACKING_URI set (in GitHub Actions, from repository secrets), runs go to DagsHub.
- Without it (local development), runs go to a local mlflow.db, viewable with mlflow ui.
- If the tracking server cannot be reached, a warning is logged and the pipeline carries on.

Credentials (MLFLOW_TRACKING_URI, MLFLOW_TRACKING_USERNAME, MLFLOW_TRACKING_PASSWORD) are stored as GitHub repository secrets, never in code.

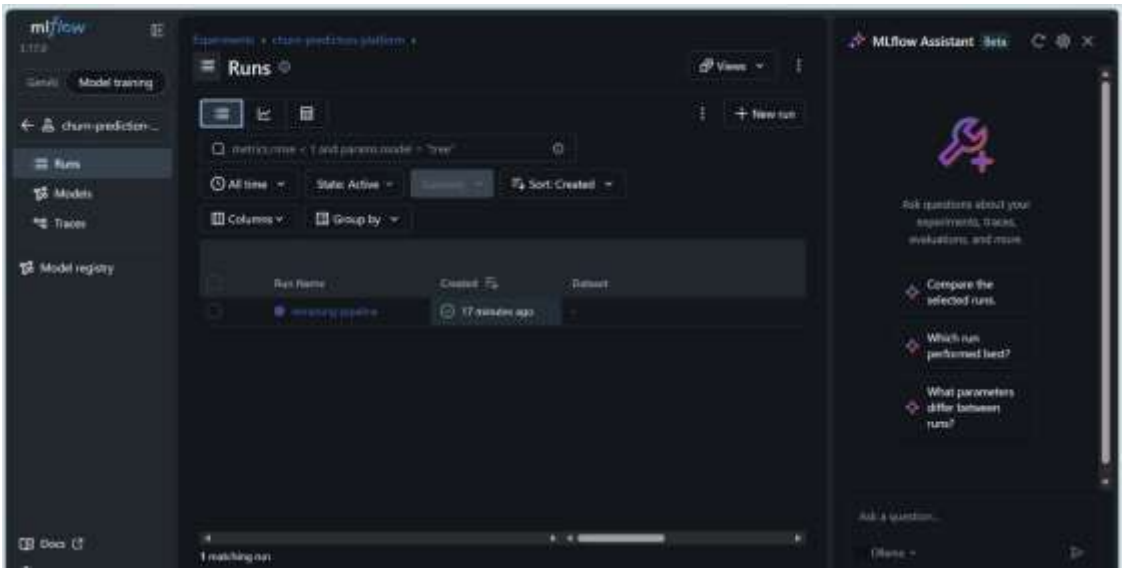


Figure 2. MLflow runs view for the churn-prediction-platform experiment

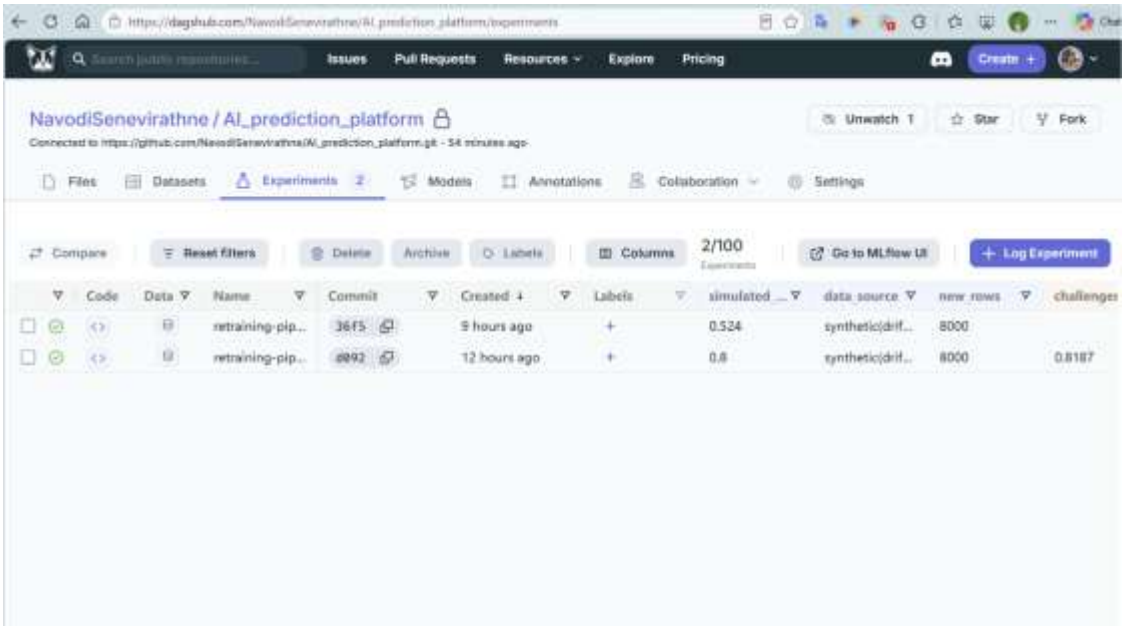


Figure 3. DagsHub experiments: each retraining run logged with its drift level, data source and challenger score

6. Real-time prediction API

The FastAPI service scores one customer in about 10 ms, validates every input, and logs every prediction so it can be monitored. The model is loaded once at start-up from models/.

Endpoints

Method	Endpoint	Purpose
POST	/predict	Score one customer
POST	/predict/batch	Score up to 1,000 customers
GET	/health	Liveness, model version, uptime (used by Render's health check)
GET	/model/info	Model metadata, metrics and version history
POST	/model/reload	Reload the model without a restart; requires the X-Admin-Token header
GET	/monitoring/summary	Traffic, average and P95 latency, predicted churn rate, probability histogram
GET	/monitoring/drift	Drift report on the last N predictions
GET	/monitoring/recent	Latest logged predictions
GET	/metrics	Prometheus-format metrics
GET	/docs	Interactive OpenAPI documentation

Example

Request:

```
POST /predict
{
 "age": 34, "tenure_months": 5, "monthly_charges": 89.5, "total_charges": 447.5,
 "num_support_calls": 4, "avg_daily_usage_gb": 6.2, "contract_type": "month_to_month",
 "internet_service": "fiber", "payment_method": "electronic_check"
}
```

Response:

```
{
 "prediction_id": "774a42f9bd16426ca0b29a2561ef8ea1",
 "churn_probability": 0.9021,
 "churn_prediction": true,
 "risk_level": "high",
 "model_version": 1,
 "latency_ms": 253.841
}
```

This response was the first request after start-up, which is slower; in local load tests over 650 requests the average latency was 9.7 ms and P95 17.7 ms.

Input validation

Pydantic schemas reject bad requests with HTTP 422 before they reach the model: an age outside 18–100, negative charges, more than 50 support calls, an unknown contract type, or a missing field.

Prediction logging

Each prediction is written to SQLite with its id, timestamp, model version, all input features, probability, predicted class and latency. Writes are thread-safe. This log feeds the monitoring endpoints and the live drift check.

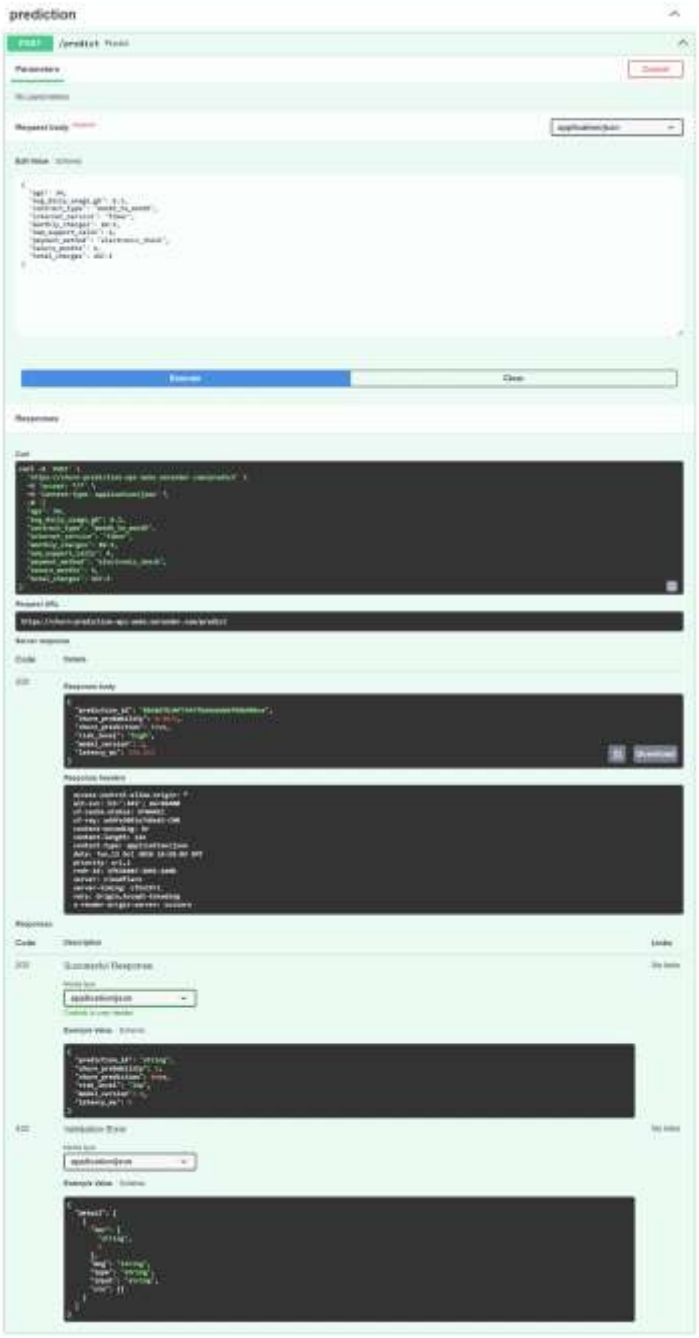


Figure 4. Interactive API docs: POST /predict returns a 98.75% high-risk churn prediction from live model v2

7. Data drift monitoring

Drift is measured per feature by comparing current data with the reference sample the production model was trained on; the dataset counts as drifted when at least 30% of features have PSI of 0.2 or more. The same code (src/monitoring/drift.py) runs in two places: on live traffic through /monitoring/drift, and inside the retraining pipeline.

Population Stability Index (PSI)

PSI is the decision metric. Numeric features are split into 10 quantile bins of the reference data; categorical features use their category shares.

$$\text{PSI} = \sum_i (a_i - e_i) \ln \frac{a_i}{e_i}$$

Here e is the share of reference rows in bin i and a the share of current rows; shares are floored at 0.0001 to avoid division by zero.

PSI	Reading
below 0.1	Stable
0.1 to 0.2	Moderate change
0.2 and above	Significant drift: feature flagged

PSI is used as the decision metric rather than p-values, because on large samples a statistical test flags even tiny, irrelevant shifts.

Statistical tests (reported alongside PSI)

- Numeric features: two-sample Kolmogorov–Smirnov test.
- Categorical features: chi-squared test on category counts.

Thresholds

Setting	Default	Environment variable
Feature drift threshold (PSI)	0.2	PSI_DRIFT_THRESHOLD
Dataset drift: share of drifted features	30%	DATASET_DRIFT_SHARE
Minimum live window for a drift check	50 predictions	(fixed)

Each report lists, per feature, the PSI, the test and its p-value, the reference and current means (numeric), and whether it drifted. It is shown on the dashboard, saved as drift_report.json in MLflow, and summarised in the GitHub Actions job summary.

After a model is promoted, its own training sample becomes the new reference. The drifted market therefore becomes the new normal: once v2 went live, traffic at drift 0.9 no longer raised an alert, while traffic resembling the original customers (drift 0.0) was flagged, with drift in 3 of 9 features (monthly_charges, num_support_calls, avg_daily_usage_gb). This confirms the model genuinely adapted to the new market rather than being retrained once.

8. Automated retraining pipeline

The pipeline (src/pipelines/retrain.py) retrains only when the data or the model's performance has changed, and promotes a new model only when it beats the current one on the same fresh data. It runs daily at 00:30 UTC (06:00 Sri Lanka time) and on demand.

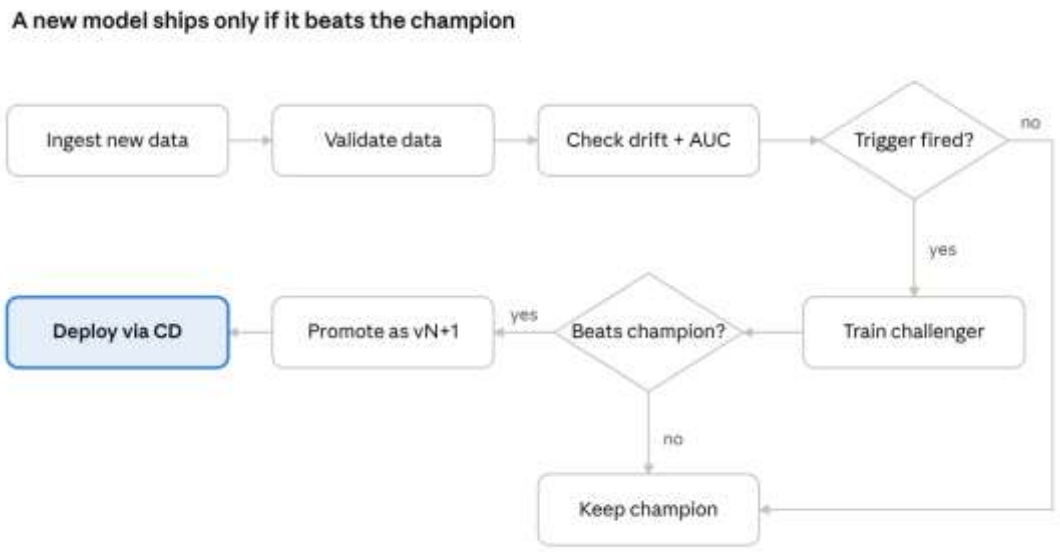


Figure 5. Retraining flow: a new model ships only if it beats the champion

Either "no" ends the run with the champion still serving; a validation failure stops the run before any decision.

Steps

Step	What happens
1. Ingest	Load 8,000 new labelled customers (a CSV, or the simulated source at the day's drift level)
2. Validate	Data-quality checks (section 3); any failure stops the run
3. Monitor	Drift report against the champion's reference sample; champion scored on a 30% fresh hold-out
4. Decide	Retrain if any trigger fires (table below); otherwise keep the champion and stop
5. Train challenger	Same hyperparameters, trained on 70% of the new data plus 50% of the reference sample
6. Compare	Champion and challenger evaluated on the same fresh hold-out
7. Promote	Only if challenger AUC ≥ champion AUC + 0.001: written to models/ as version N+1

Mixing in part of the reference sample keeps the challenger from forgetting older customer behaviour while it learns the new one.

Retraining triggers

Trigger	Default	Environment variable
Dataset drift detected	≥ 30% of features with PSI ≥ 0.2	DATASET_DRIFT_SHARE, PSI_DRIFT_THRESHOLD
Champion AUC on new data too low	below 0.80	MIN_ACCEPTABLE_AUC
Champion AUC dropped since	more than 0.03	MAX_AUC_DROP

training		
Manual force	--force flag or workflow input	—

Outputs of every run

- reports/latest_retrain_report.json and one line appended to reports/retrain_history.jsonl
- An MLflow run with parameters, champion and challenger metrics, the drift report and, when promoted, the registered model
- A GitHub Actions job summary: drift table, metrics and decision
- Step outputs promoted and new_version, which the workflow uses to start deployment

Run locally

```
python -m src.pipelines.retrain              # drift level from today's date
python -m src.pipelines.retrain --drift 0.8  # simulate a strong shift
python -m src.pipelines.retrain --force      # retrain even without a trigger
python -m src.pipelines.retrain --data new.csv # use real labelled data
```

9. CI/CD

Three GitHub Actions workflows test every change, deploy every approved change, and retrain on a schedule; a promoted model reaches production through the same tested CD path as a code change.

Workflow	Trigger	Jobs and steps
CI (ci.yml)	Every push and pull request	ruff lint → 17 pytest tests → quality gate (production model ROC-AUC ≥ 0.75 on fresh data) → build API and dashboard images → run the API container and smoke-test /health and /predict
CD (cd.yml)	Push to main, manual, or called by retraining	Tests again → build and push images to GHCR → call the Render deploy hooks → poll /health until the expected model version is live (up to 20 minutes) → production smoke test
Automated retraining (retrain.yml)	Daily cron 30 0 * * *, or manual with a drift level and a force option	Run the pipeline → upload the report → if promoted, commit models/ and reports/ as mlops-bot → call CD

Commits made with the workflow's GITHUB_TOKEN do not trigger other workflows, so the retraining workflow calls CD directly as a reusable workflow (workflow_call).

Image tags

Each CD run pushes the API image as api:latest, api:<git-sha> and api:model-v<N>, and the dashboard image as dashboard:latest and dashboard:<git-sha>. Old tags stay in the registry, so any model version can be redeployed.

Repository settings

Type	Name	Purpose
Secret	MLFLOW_TRACKING_URI	DagsHub MLflow endpoint
Secret	MLFLOW_TRACKING_USERNAME	DagsHub user
Secret	MLFLOW_TRACKING_PASSWORD	DagsHub access token
Secret	RENDER_API_DEPLOY_HOOK	Deploy hook of the API service
Secret	RENDER_DASHBOARD_DEPLOY_HOOK	Deploy hook of the dashboard service
Variable	API_URL	Public API URL used to verify deployments
Permission	Workflow permissions: read and write	Lets the bot commit promoted models and push images

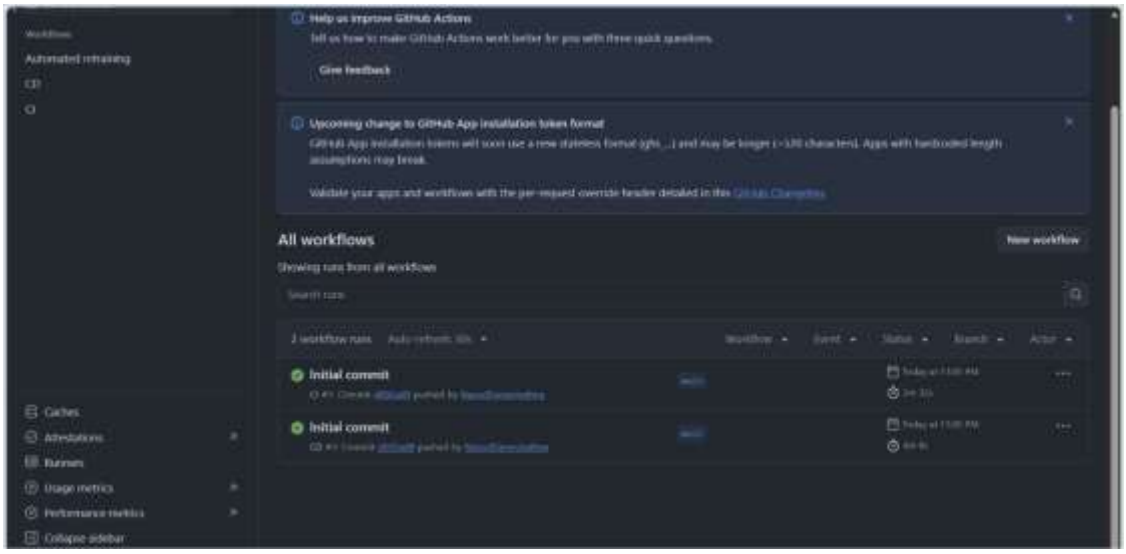


Figure 6. GitHub Actions: first CI and CD runs passed on the initial commit

10. Deployment

Both services run as Docker containers on Render's free tier in the Singapore region, and are redeployed only through CD's deploy hooks.

Docker images

Image	Dockerfile	Contents	Port
API	Dockerfile	python:3.12-slim, API requirements, src/, models/	$PORT (default 8000)
Dashboard	Dockerfile.dashboard	python:3.12-slim, dashboard requirements, src/, dashboard/	$PORT (default 8501)

Both images install dependencies in a separate cached layer, run as a non-root user, and read the port from $PORT. The API image has a HEALTHCHECK on /health. Core libraries are pinned exactly (scikit-learn 1.9.1, pandas 3.0.5, numpy 2.5.3) so a model trained in CI loads identically in the container.

Render services

Setting	API	Dashboard
Service name	churn-prediction-api	churn-monitoring-dashboard
Dockerfile path	./Dockerfile	./Dockerfile.dashboard
Instance type	Free	Free
Health check path	/health	—
Auto-deploy	Off	Off
Environment variables	ADMIN_TOKEN	API_URL = the API's public URL

Render keeps every deploy; a bad model can be rolled back with one click to the previous deploy.

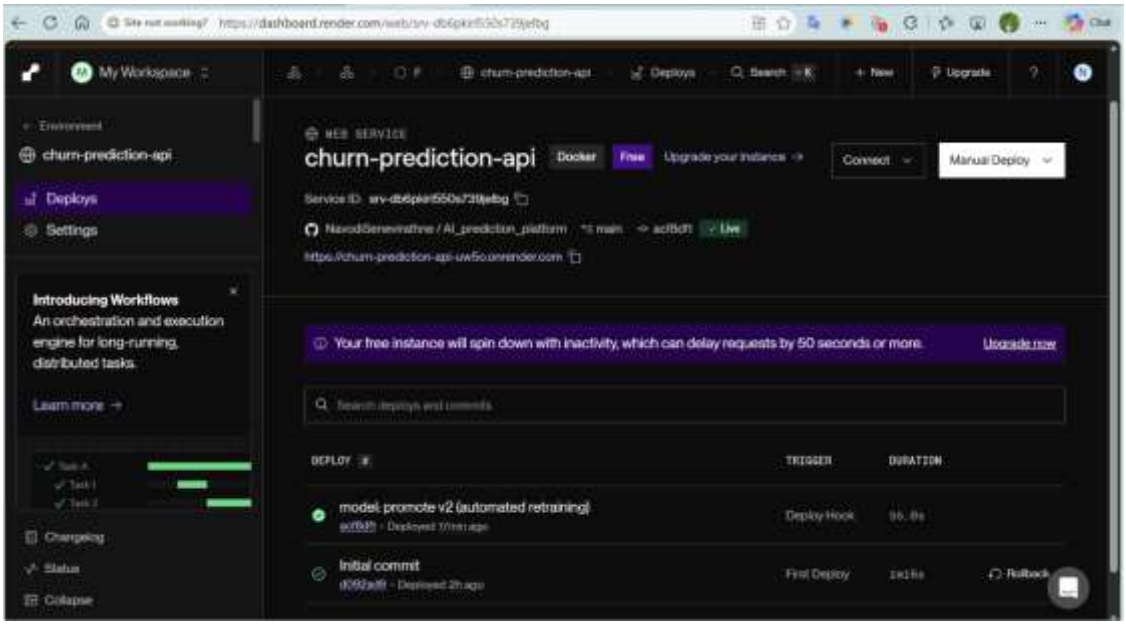


Figure 7. Render deploys: v2 deployed by the deploy hook, the initial v1 deploy kept for rollback

Running the whole platform locally

```
docker compose up --build                    # API :8000, dashboard :8501, MLflow :5000
docker compose --profile pipeline run --rm retrain   # retrain against the local MLflow
docker compose --profile pipeline run --rm simulator # stream simulated traffic
```

11. Monitoring dashboard

The Streamlit dashboard (dashboard/app.py) reads everything from the API's REST endpoints, so it always shows the live system, including which model version is serving.

Tab	Shows
Live traffic	Total and in-window predictions, average and P95 latency, predicted churn rate; requests, churn rate and latency per minute; probability histogram; latest predictions
Data drift	Dataset drift alert, PSI per feature (chart and table with test, p-value, reference and current means)
Model & retraining	Current model metrics, version history chart (ROC-AUC and F1 by version), full metadata
Try a prediction	Form to score one customer in real time

The sidebar sets the monitoring window (15 minutes to 7 days) and the drift window (last 100–5,000 predictions). It also has a traffic simulator that sends 50–1,000 simulated customers at a chosen drift level, used to demonstrate drift detection live.

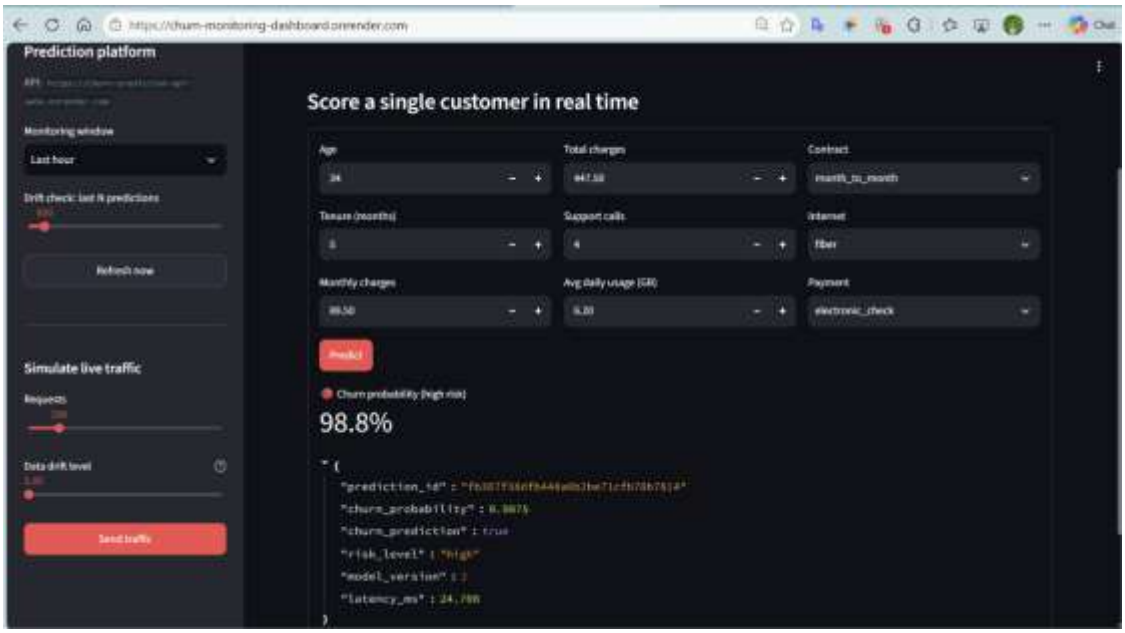


Figure 8. Try a prediction: a new month-to-month customer with 4 support calls is scored 98.8% (high risk) by model v2

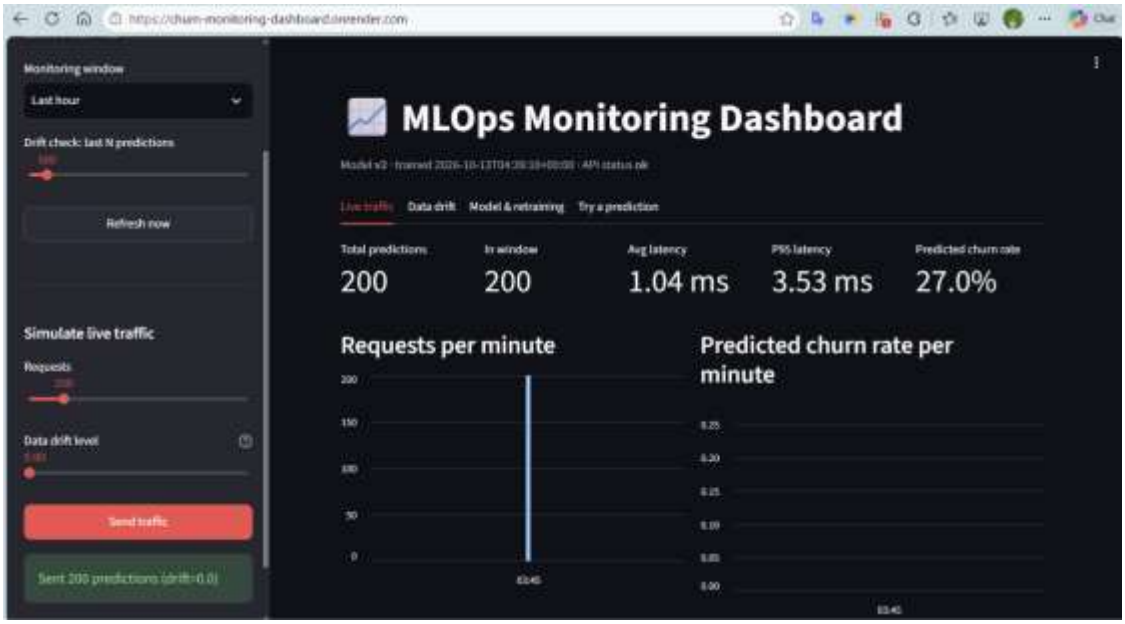


Figure 9. Live traffic: 200 normal customers scored, average latency about 1 ms, predicted churn rate 27%

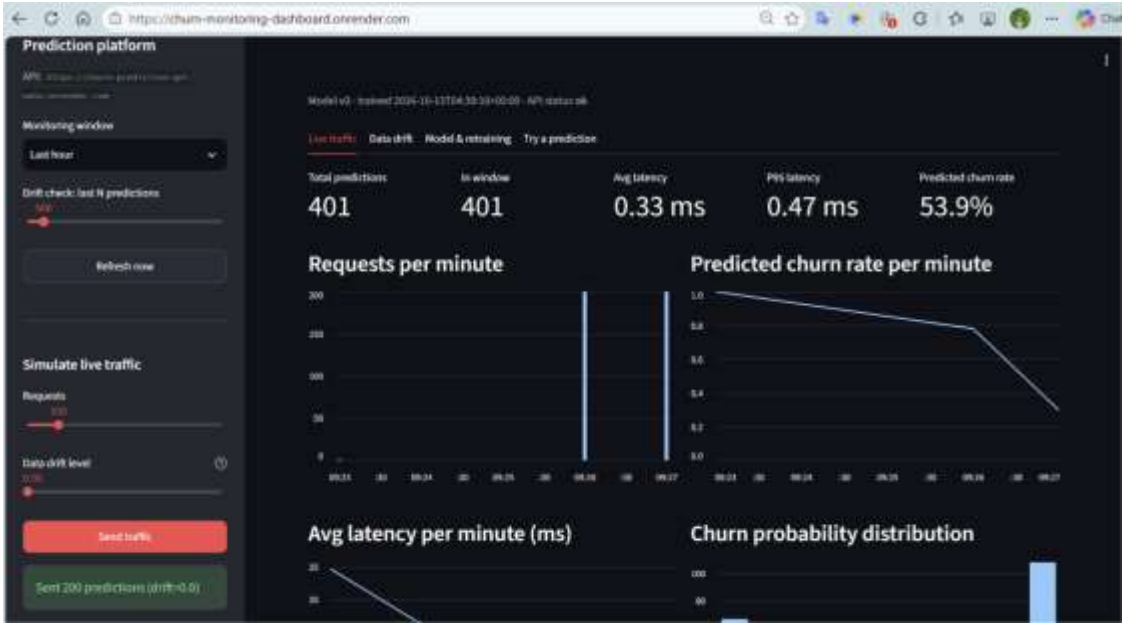


Figure 10. After drifted traffic arrives, the predicted churn rate rises from 27% to 54%, tracked minute by minute

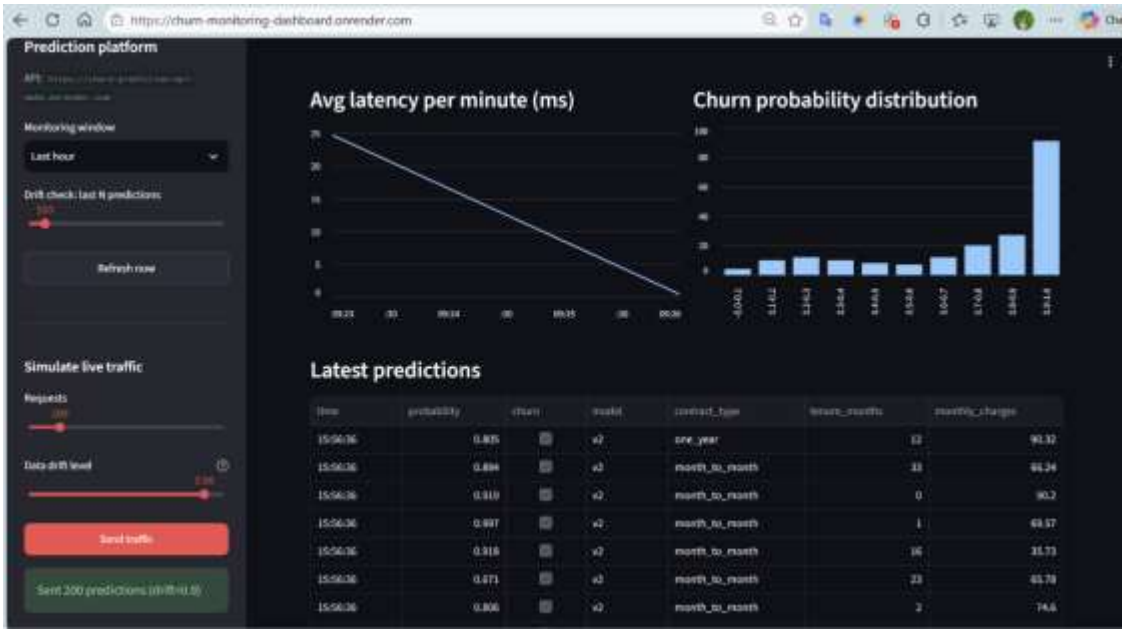


Figure 11. Drifted market: higher bills and more support calls push most predictions into the 0.9 to 1.0 range

12. Testing

17 automated tests (pytest, about 5 seconds) run on every push in CI and again before every deployment; the code is also linted with ruff.

Area	File	What is checked
Data	tests/test_ml.py	Generator schema, reproducibility with a seed, realistic churn rate, rejection of invalid drift levels
Validation	tests/test_ml.py	Unknown categories and missing columns are caught
Drift	tests/test_ml.py	PSI is small for the same distribution and large for a shifted one; drift report is correct and JSON-serialisable
Model	tests/test_ml.py	Quality gate: ROC-AUC above 0.8 on held-out data
Pipeline	tests/test_ml.py	Champion kept without drift; retraining triggered and challenger evaluated with drift; date-based drift schedule
API	tests/test_api.py	Health, single and batch prediction, input validation (422), high-risk customer scores above a loyal one, monitoring summary, drift endpoint, recent predictions, Prometheus metrics, admin token on reload

CI adds two checks beyond pytest: the committed production model must reach ROC-AUC 0.75 or more on 5,000 fresh customers, and the built API container must answer /health and /predict.

```
pytest -v
ruff check .
```

13. Results

The full loop ran end to end without manual steps: drifted traffic was detected, a challenger beat v1 (ROC-AUC 0.8669 against 0.8538 on the same new data), and v2 was deployed and verified live in 4 minutes 40 seconds.

Retraining decision

- 8,000 new customers at drift 0.8.
- Dataset drift: 3 of 9 features drifted (33%, above the 30% threshold): monthly_charges PSI 1.01, num_support_calls 1.11, avg_daily_usage_gb 0.60.
- Champion v1 ROC-AUC on the new data 0.8538, a drop of 0.011 from training, so drift was the only trigger.
- Challenger ROC-AUC 0.8669 on the same hold-out, so it was promoted to v2.

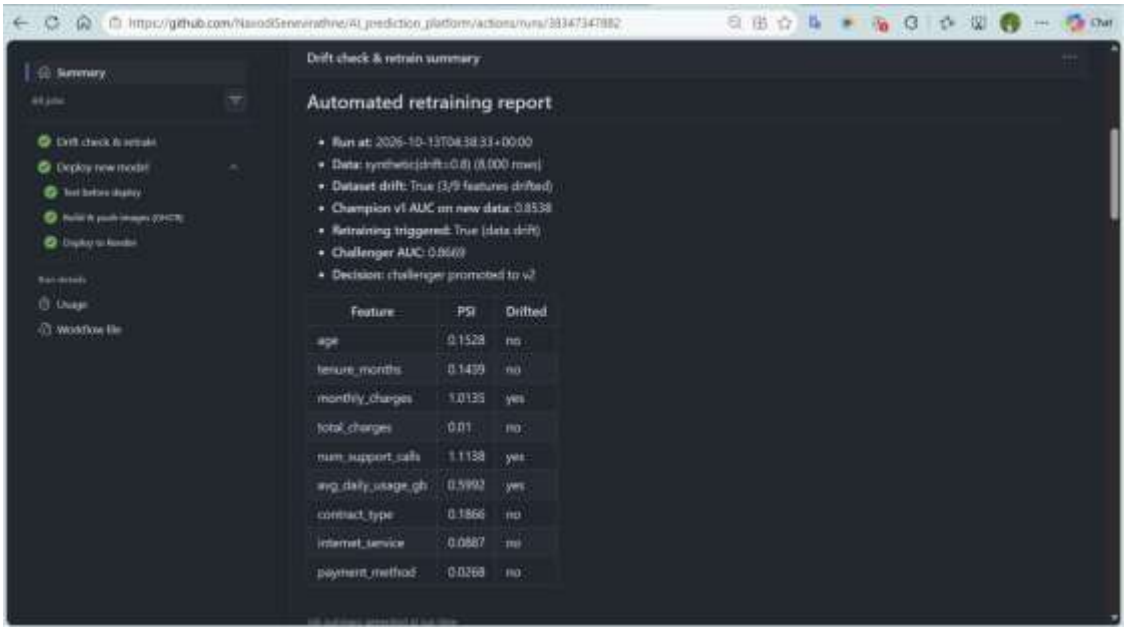


Figure 12. Retraining report from the GitHub Actions job summary

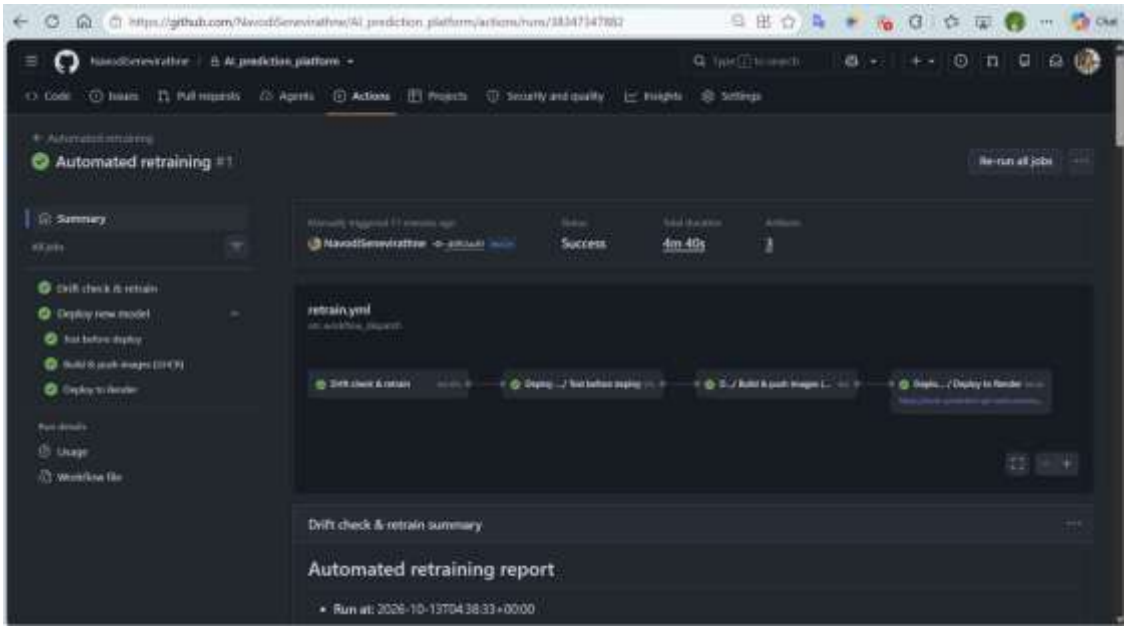


Figure 13. All four jobs of the retraining run passed in 4 minutes 40 seconds

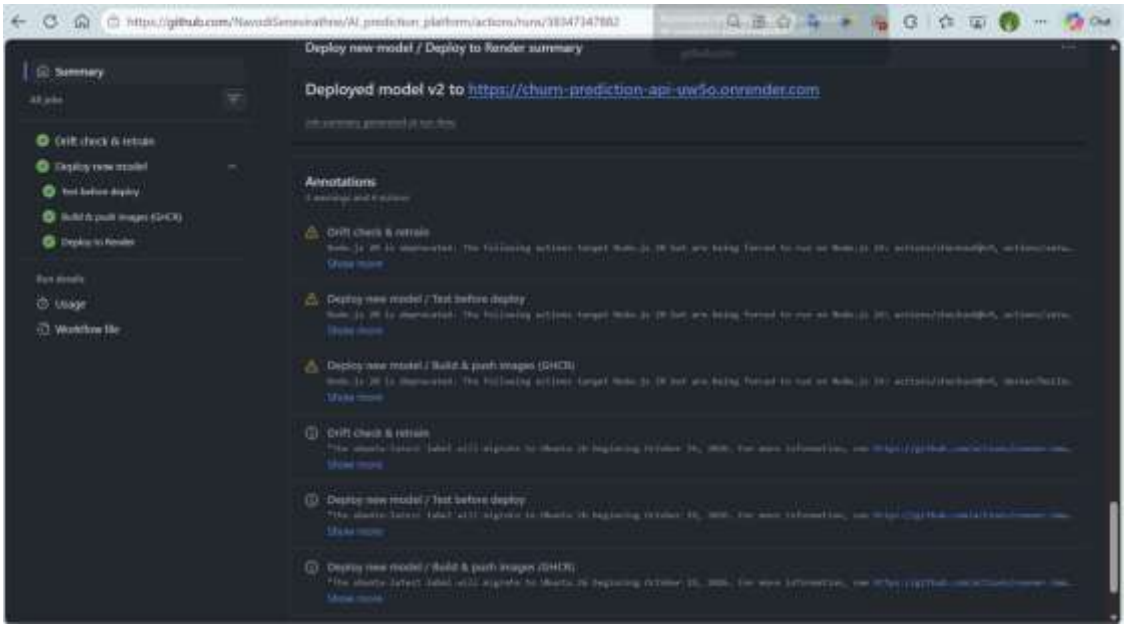


Figure 14. Deploy step confirmed model v2 live at the production URL

Model versions

Version	Created	Reason	ROC-AUC	Evaluated on
v2 (live)	13 Oct 2026	Data drift; AUC 0.8538 → 0.8669	0.8669	2,400 new customers at drift 0.8
v1	12 Oct 2026	Initial training	0.8644	4,000 customers at drift 0

The two ROC-AUC values come from different evaluation sets, so the fair comparison is the one the pipeline makes: both models on the same new data. The live version history is at https://churn-prediction-api-uw5o.onrender.com/model/info.

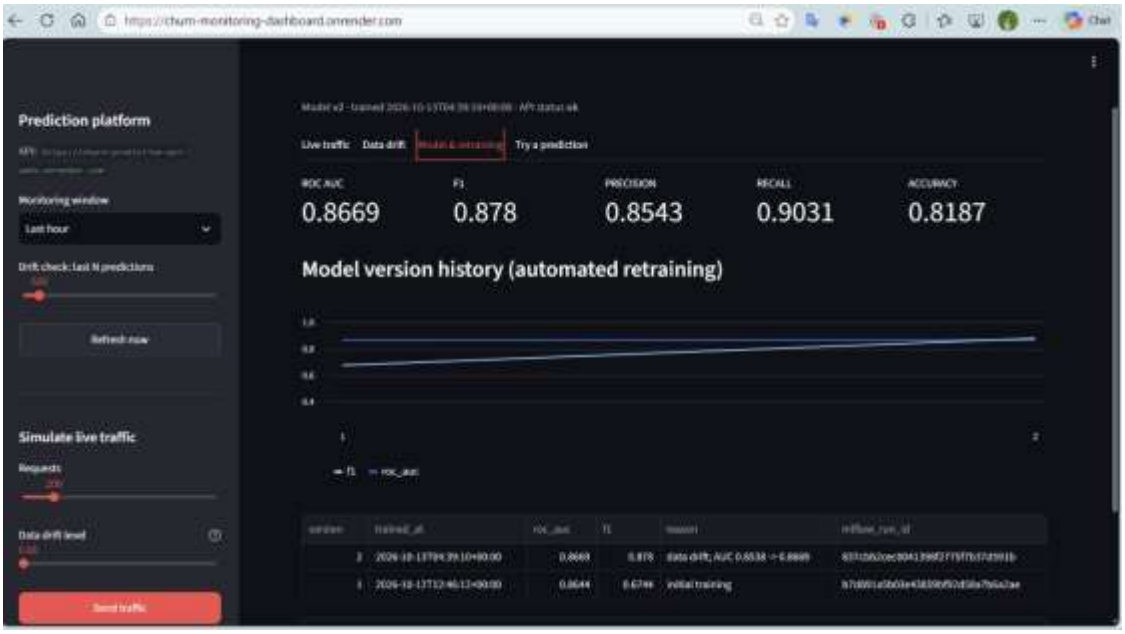


Figure 15. Model & retraining tab: v1 (initial training, AUC 0.864) and v2 (data drift, AUC 0.867) with their MLflow run ids

Other checks

- Without drift, the pipeline kept the champion (test run: 0 of 9 features drifted, champion AUC 0.8714, "no retraining needed").
- With strong drift (1.0), the pipeline found 6 of 9 features drifted, an AUC drop of 0.056 below the 0.80 floor, and promoted the challenger (AUC 0.8406).
- Both Docker images built in CI and CD, with the API container passing its smoke test.

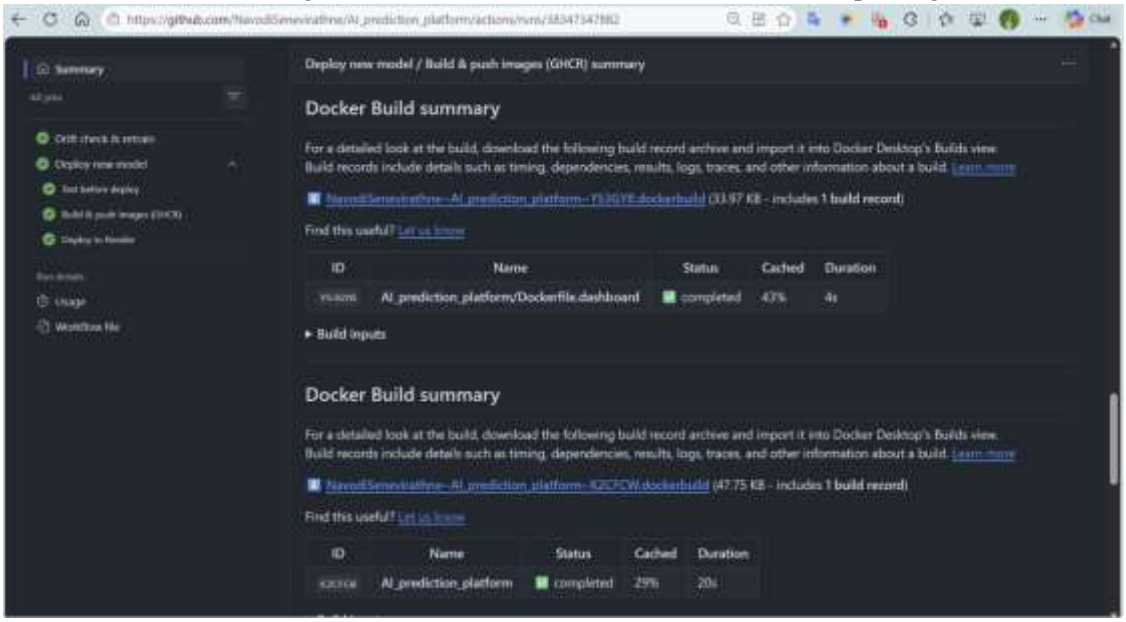


Figure 16. Docker build summary for the API and dashboard images

14. Limitations and future work

The platform is complete for its scope; the main limits come from free-tier hosting and from using a simulated labelled data feed.

Limitations

Limitation	Effect	Production fix
Simulated data source	Retraining learns from freshly simulated labelled data, not from the API's logged traffic	Read labelled outcomes from the CRM, or add a /feedback endpoint that joins true churn outcomes to logged predictions
Delayed labels	Real churn is known only weeks after a prediction	Retrain on a rolling window of predictions whose outcomes have arrived; monitor drift in the meantime
Free-tier sleep	Services sleep after 15 minutes idle; the first request takes about a minute	Paid instance, or a keep-alive ping
Ephemeral prediction log	The SQLite log resets when the API container restarts or redeploys	Managed PostgreSQL or a data warehouse
Small free instance	512 MB memory; heavy bursts of traffic can return 502 while the instance restarts	Larger instance, several Uvicorn workers or replicas
No API authentication	Anyone with the URL can call /predict	API keys or OAuth, plus rate limiting

Future work

- /feedback endpoint so retraining uses real logged traffic with its true outcomes.
- Alerts (email or Slack) when dataset drift is detected or a model is promoted.
- Grafana dashboards on the existing /metrics endpoint.
- Canary or shadow deployment: send a share of traffic to the challenger before full promotion.
- Hyperparameter search (for example Optuna) inside the retraining pipeline.
- Message queue (Kafka or Pub/Sub) for high-volume streaming predictions.

Appendix

A. Repository structure

```
src/
  config.py                features, paths, drift and retraining thresholds
  data/generate.py         simulated customer data with controllable drift
  training/model.py        pipeline, evaluation, production model registry
  training/tracking.py     MLflow integration (safe when offline)
  training/train.py        initial (bootstrap) training
  monitoring/drift.py      PSI, KS and chi-squared drift detection
  pipelines/retrain.py     automated champion/challenger retraining
  api/main.py              FastAPI app and endpoints
  api/schemas.py           request and response schemas
  api/prediction_store.py  prediction log (SQLite)
dashboard/app.py           Streamlit monitoring dashboard
scripts/simulate_traffic.py streams simulated traffic to the API
models/                    current production model, metadata, reference sample
reports/                   retraining reports written by the pipeline
tests/                     17 unit and integration tests
.github/workflows/         ci.yml, cd.yml, retrain.yml
Dockerfile                 API image
Dockerfile.dashboard       dashboard image
docker-compose.yml         API, dashboard and MLflow locally
render.yaml                Render blueprint (optional)
requirements/              pinned dependencies per component
```

B. Running locally (Windows PowerShell)

```
py -3.13 -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest                              # 17 passed
uvicorn src.api.main:app --reload       # http://localhost:8000/docs
# in a second terminal
python scripts/simulate_traffic.py --n 300
python scripts/simulate_traffic.py --n 300 --drift 0.9
$env:API_URL = "http://localhost:8000"
streamlit run dashboard/app.py          # http://localhost:8501
# retraining and experiment tracking
python -m src.pipelines.retrain --drift 0.8
mlflow ui --backend-store-uri sqlite:///mlflow.db --workers 1   # http://localhost:5000
```

C. Technology stack

Area	Technology
Language	Python 3.12 (containers), 3.13 (local)
Machine learning	scikit-learn 1.9.1, pandas 3.0.5, numpy 2.5.3, scipy 1.18.1
Serving	FastAPI 0.143, Uvicorn 0.53, Pydantic 2.13
Dashboard	Streamlit 1.65
Experiment tracking	MLflow 3.17 on DagsHub
Containers	Docker, GitHub Container Registry
CI/CD and orchestration	GitHub Actions
Hosting	Render (free web services, Singapore)
Quality	pytest 9.1, ruff 0.17